

Migrating Deprecated API to Documented Replacement: Patterns and Tool

Yaoguo Xi

School of Computer Science, Fudan University
Shanghai Key Laboratory of Data Science, Fudan
University
Shanghai, China

Yukun Gui

School of Computer Science, Fudan University
Shanghai Key Laboratory of Data Science, Fudan
University
Shanghai, China

Liwei Shen

School of Computer Science, Fudan University
Shanghai Key Laboratory of Data Science, Fudan
University
Shanghai, China

Wenyun Zhao

School of Computer Science, Fudan University
Shanghai Key Laboratory of Data Science, Fudan
University
Shanghai, China

ABSTRACT

Deprecation is commonly leveraged to preserve the backwards compatibility in API clients during API evolution. Client developers then usually encounter migration tasks when relying on a new version of API. Manually migrating a deprecated API to its replacement may be tedious and time-consuming. Existing research has focused on the requirements of automated migration however has limitations on the consistency and granularity of the generated migration pathway. In this paper, we propose an approach for migrating deprecated APIs to their replacements which are explicitly documented in the code documentation. A migration pattern including the migration type and the migration strategy is first introduced to describe the difference and the code change from a deprecated API to its replacement. An automated process is then designed and implemented to construct the pattern for a specific deprecated API by reviewing the code documentation and analyzing the API source code. Based on the patterns, a tool named *DAAMT* is developed to locate deprecated APIs and to provide migration suggestions for API clients. An experimental study on open-source Java projects and a user study is performing utilizing the approach. The results show that three-fourth of the deprecated APIs with documented replacements involved in the projects can be automatically migrated. In addition, the *DAAMT* tool can improve the efficiency and accuracy of the tasks compared with the manual migration.

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**;
Software libraries and repositories; *Maintaining software*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Internetware '19, October 28–29, 2019, Fukuoka, Japan

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7701-0/19/10...\$15.00

<https://doi.org/10.1145/3361242.3361246>

KEYWORDS

Deprecated API, API Migration, Migration Pattern.

ACM Reference Format:

Yaoguo Xi, Liwei Shen, Yukun Gui, and Wenyun Zhao. 2019. Migrating Deprecated API to Documented Replacement: Patterns and Tool. In *Internetware '19: Proceedings of the 11th Asia-Pacific Symposium on Internetware (Internetware '19), October 28–29, 2019, Fukuoka, Japan*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3361242.3361246>

1 INTRODUCTION

An application programming interface (API) prescribes the expected behavior encapsulated in a library or a framework, for example the standard Java Development Kit (JDK). Software developers tend to use APIs in modern paradigms to avoid repetitive efforts [13] and to enhance the reuse in software systems [8]. However, the API clients may be broken when the systems rely on newly released APIs in which the referenced methods' signature has changed or even the methods have been removed. As an acknowledged practice, *Deprecation* is proposed by API developers to preserve the backwards compatibility in API clients which follows a deprecate-replace-remove cycle [11]. The cycle means that certain features in APIs are obsolete and they are replaced by updated elements before removed in a future release [16]. Under the mechanism, developers are warned and encouraged to migrate the deprecated APIs to their replacements in order to benefit from improved API elements [1], i.e., enhancement on functionality, performance or security [11].

The migration can be performed by developers manually in different means such as reading and understanding the replacement message in the documentation, retrieving the code examples of the replacement from online repositories, or even reviewing the source code of the deprecated API and its replacement. To reduce developers' efforts, recent research has proposed approaches to provide suggestions for API migration typically depending on the code documentation [15, 18] which has been evaluated more effective than the historical code-change information [10]. However, the migration pathway obtained by these methods is either inconsistent with the replacement message [15] or does not resolve the replacement problem at a more granular API member level [18]. The fact implies the lack of an approach to provide automated migration strategies

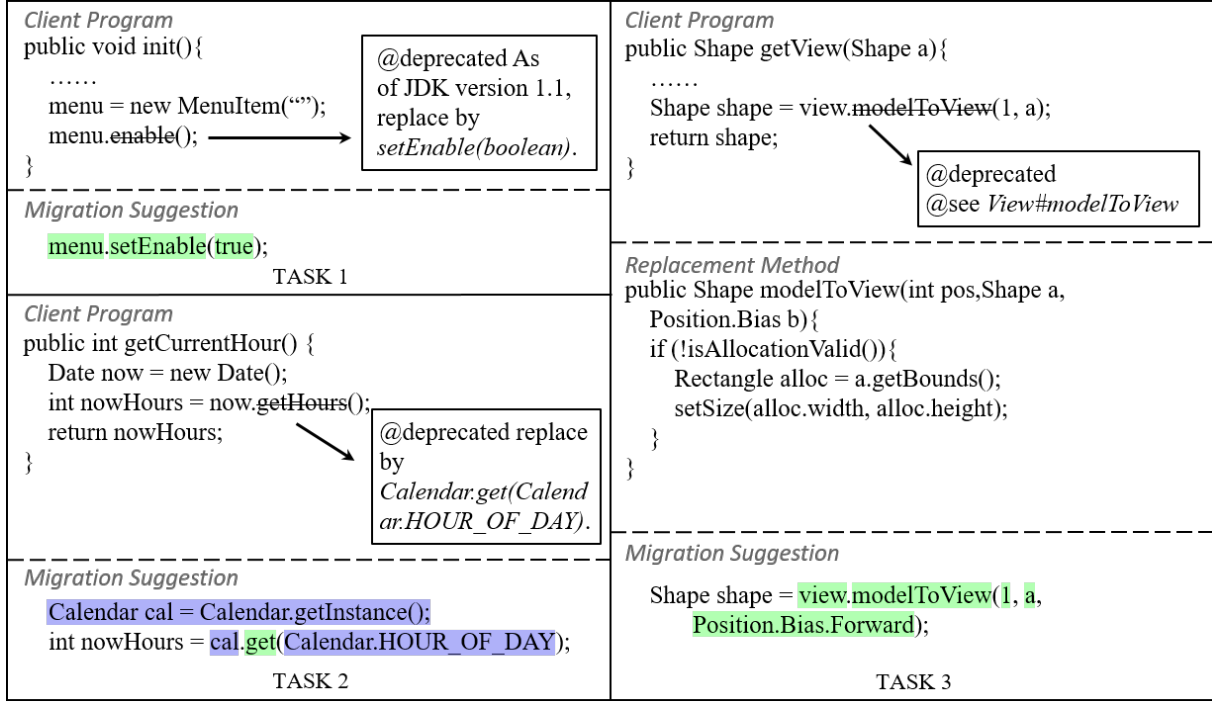


Figure 1: Example Deprecated API Migration Tasks.

more accurately with fully consideration of code documentation and source code.

In this paper, we propose an approach for migrating deprecated APIs to their replacements which are explicitly documented in the code documentation. We first introduce API migration pattern which presents the migration type and the migration strategy from a deprecated API to its replacement. By reviewing the code documentation of the deprecated APIs with documented replacement, we identify the type denoting the difference between an API and its replacement on the signature level, and totally conclude eight migration types. On the other hand, we synthesize the migration strategy by analyzing the source code covering the migration operations of code changes for the API caller, the API name, the return type and the parameters. Each operation is further assigned with a confidence level according to the association between the elements of the deprecated API and its replacement. Based on the patterns, a tool named *DAAMT* (*Deprecated API Assisted Migration Tool*) is developed to locate deprecated APIs and to provide migration suggestions for API clients. The tool leverages different background color for the migrated elements with different confidence level. In addition, annotations may also be appended for the elements whose migration strategies cannot be directly derived.

We constructed the deprecated API migration patterns contained in five java-based libraries as the base for the tool evaluation including an experimental study and a user study. In the experimental study, the patterns were applied to a set of open-source projects from github and the migration suggestions for the deprecated APIs involved were given out by the tool. The results show that three-fourth of the deprecated APIs with documented replacements can

be automatically migrated, while the others are commented by migration suggestions mostly indicating the parameters of the replacement API. In the user study, we invited users to solve specific migration tasks leveraging the tool. The results show that *DAAMT* can improve the efficiency and accuracy of the tasks compared with the manual migration.

The remainder of this paper is organised as follows. Section 2 presents motivation examples for automated deprecated API migration. Section 3 presents the API migration patterns, covering its definition and the construction process by analyzing the code documentation as well as the source code. Section 4 describes *DAAMT*. Section 5 evaluates the approach by two studies. Section 6 reviews the related work, followed by our conclusion in Section 7.

2 MOTIVATION EXAMPLES

Developers may encounter deprecated API migration tasks when relying on a new version of API. Without the support of automated tools, they have to manually determine the migration pathways and perform the corresponding code changes.

Figure 1 illustrates three tasks of deprecated API migration in Elisa's work plan. In the first task, Elisa read the replacement message of the API *MenuItem.enable* and understood the behavior of the replacement API *MenuItem.setEnabled(boolean)*. The migration was easy for preliminary experienced developers such as Elisa that she could directly decide to use the new API with the right parameter. The second task might bring difficulties to Elisa because the replacement message dose not guarantee the successful compilation of the client program. In this case, the deprecated API *Date.getHours()* can be replaced by *Calendar.get(Calendar.HOUR_OF_DAY)* which is

Table 1: Deprecated API Migration Strategy.

Element	Condition	Operation	Confidence
Caller	Traceable	setCaller(caller,statements)	H
	Instantiable	createCaller(className,statement)	M
	Unknown	annotate(annotations)	L
Name	Change	rename(rAPIName)	H
Return Type	Traceable	convertType(statements)	H
	Convertible	convert(dType,rType)	M
	Unknown	annotate(annotations)	L
Parameter	Creation	addParam(rParam,statements/expression)	H
	Deletion	removeParam(dParam)	H
	Change	updateParam(dParam,rParam,statements/expression)	H
	WeakMap	setParam(dParam,rParam)	M
	Unknown	annotate(annotations)	L

documented. The replacement API is not a static method and *Calendar* is an abstract class. Elisa then turned to Google Search using the keywords “*replace deprecated date.getHours*” and learnt from a post in Stack Overflow¹ which shows the correct usage of this replacement. The third task might cost Elisa more efforts since the parameter list of the replacement API is not explicitly specified. Furthermore, there was no result from the search engine related to this migration pathway. Under these circumstances, Elisa had to locate the replacement API, read the source code of the API and understand how the new parameter list should be synthesized. In this example, she finally found that the replacement API has one more parameter (*Position.Bias.Forward* as a constant) than the deprecated API but preserves the same API name.

The example tasks represent API migrations of varying difficulty mainly depending on the developer’s experience and the quality of the documentation. However, regardless of the difficulty, all tasks require a certain amount of efforts for developers.

Using the *DAAMT* tool, Elisa can have a completely different experience. She can take the project as the input of the tool, and obtain the migration suggestions automatically for the deprecated APIs that have been analyzed beforehand. Subsequently, Elisa is able to evaluate the suggestions and choose whether to perform the replacement. The lower part of each sub-figure illustrates the tool interface when coping with the deprecated APIs. The tool leverages different background color to indicate the different confidence level for the API elements. For example, green means the highest confidence that the correctness of the migration for the return type or a parameter can be guaranteed. Blue means the medium confidence that the suggestion should be reviewed by developers. Compare with the manual migration, *DAAMT* reflects its advantages in terms of efficiency and accuracy.

3 API MIGRATION PATTERN

In this section, we first introduce the *API Migration Pattern* and its constituents. Then we design and implement an automated process to construct the pattern for a deprecated API with documented replacement. Patterns contained in five java-based class libraries are generated and listed leveraging the process.

3.1 Definition

An *API Migration Pattern (MP)* is a five-tuple that contains the relationship between a deprecated API and its replacement API, which can be defined as: $MP = \langle APIVersion, dAPI, rAPI, MT, MS \rangle$

APIVersion indicates the version on which the deprecated API is annotated, while the replacement API is specified. *dAPI* is the fully qualified name of the deprecated API to be migrated, combined by the identifier of the package and the class. *rAPI*, relatively, is the fully qualified name of the replacement API. *MT* represents the Migration Type from the deprecated API to the replacement. It clarifies the difference between the signatures of the API pair (i.e., the deprecated API and the replacement API) and concludes the difference into several categories. *MS* represents the Migration Strategy from the deprecated API to the replacement. It contains the operations of the code changes when migration is performed.

We further define *MT* as a tuple $\langle DiffType, (dRType, rRType), (dParams, rParams) \rangle$ in which the return type and the parameter list of the APIs are mapped separately. On the signature level, the return type of the API can be changed or consistently preserved. We identify the two situations as *DR (Different Return Type)* and *SR (Same Return Type)*. On the other hand, the change of the parameter list in the signature is divided into *PA (Parameter Addition)*, *PR (Parameter Reduction)*, *PC (Parameter Change)* and *PU (Unchanged Parameter List)*. The difference type (*MT*) is thus composed of the categories from the two dimensions, indicating the change in signature from the deprecated API to its replacement. It is worth noting that the mapping from the API names has been described in the migration pattern, therefore it is neglected in the difference type in sake of simplicity. In this case, eight types are synthesized as $\{DR_PA, SR_PA, DR_PR, SR_PR, DR_PC, SR_PC, DR_PU, SR_PU\}$.

MS contains the migration operations on the source code level from the dimensions of the API caller, the API name, the return type and the parameter list when migrating the deprecated API to its replacement. Except the possible modification on the signature, the change of the API caller means to update the object that calls the deprecated API. Table 1 shows the structure of a migration strategy covering the four dimensions. In each dimension, there are different operations depending on the conditions derived from the analysis of the deprecated API and the replacement. For example, *Traceable* in *Caller* means that we can determine the caller of the replacement API from the source code of the deprecated API and

¹<https://stackoverflow.com/questions/14485951/what-should-i-use-instead-of-the-deprecated-date-gethours>

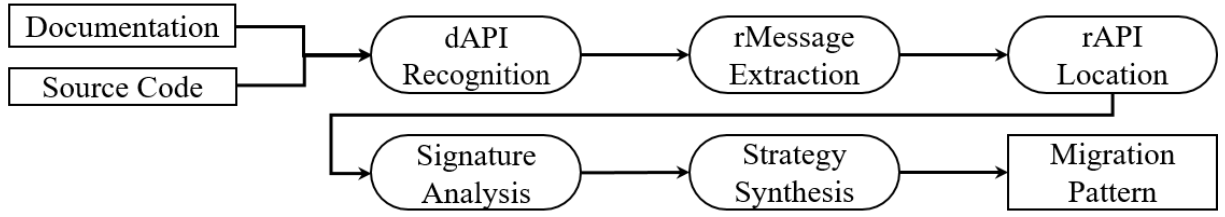


Figure 2: Overview of the Migration Pattern Construction Process.

the methods it calls. *Instantiable* means that the class to which the replacement API belongs can create an instances by mechanisms such as *constructor* or *getInstance*. *Change in Parameter* means that a parameter in *dParams* is converted to a parameter in *rParams* after assignment or type conversion operations in the source code. *WeakMap in Parameter* means that we can only recognize the relationship between the parameters of the API pair from the data types and declaration names. In addition, each operation is attached with a confidence level as listed in the table. The value of the confidence is used to suggest the credibility of the operations we generate, which also has an impact on how the migration tool is displayed. In detail, when the operation can be directly obtained from the analysis result of the signature and source code of the API pair, the operation is assigned as the *H (High)* value. When the migration operation can be completed by techniques such as textual matching other than the relationship fact of the source code, it is assigned as the *M (Medium)* value. When the change operation cannot be derived in any way, the *L (Low)* value is attached and the operation is always a natural language annotation denoting the migration target.

3.2 Process of Constructing Patterns

Figure 2 presents the process of constructing the migration pattern for a deprecated API. Rounded Rectangles are steps while rectangles are artifacts. The process takes the documentation and the source code as input while the migration pattern as output. In the process, *dAPI Recognition* aims to recognize the deprecated APIs from source code and *rMessage Extraction* serves to extract the replacement messages from the documentation. The replacement APIs are subsequently located from the source code based on the replacement message in the *rAPI Location* step. Having the API pair, *Signature Analysis* compares the signatures and generates the migration type of the pattern. *Strategy Synthesis* supplements the pattern with the migration strategy according to the migration type.

dAPI Recognition. There are two ways to deprecate a method in Java, i.e., annotating the method with *@Deprecated* or using the Javadoc tag *@deprecated* in the documentation. We consider both situations at the same time. On one hand, we use *JavaParser*² to parse the source code and determine which methods contain the *@Deprecated* annotation. On the other hand, we analyze the documentation description of all methods and separate the methods

using *@deprecated* in its documentation. An API is deemed to be deprecated if one of the above conditions is met.

rMessage Extraction. Replacement messages follow specific description patterns (e.g., *replaced by {@link ...}*). We summarize these patterns similar with those listed in [3, 18]. The Mapping between the deprecated API and its replacement message can be built by searching for these patterns in the documentation. This process also deals with some special conditions. For example, a deprecated API *JComponent.hide()* in JDK1.8 does not have the replacement message but the method *hide()* in *Component* which is the base class of *JComponent* has the replacement message.

rAPI Location. Replacement message usually consists of three parts, i.e., the class which the replacement API belongs to, the replacement API name and the number and type of parameters for the replacement. We can easily locate the replacement APIs in the source code when the message is comprehensive. However, the replacement message is often incomplete (e.g., missing fully qualified package like *setScrollMode()* and missing parameters like *View#modelToView*) which brings difficulty for directly locating the replacements. For this problem, we put forward the following strategies to solve it automatically. First, for the case of missing classes, we search from the class to which the deprecated API belongs and its base classes until we find the method with the same name and parameters as the replacement message. Second, for the case of missing parameters, we search the method by class name and method name. If multiple matches are found, we choose the one with the most similar method parameters with the deprecated API parameters. Note that there may be a one-to-many relationship between a deprecated API and its replacements as long as the message specifies this diversity.

Signature Analysis. We analyze the API signatures mainly from the perspective of *Return Type* and *Parameters*. For *Return Type*, if the data types of the APIs return value are the same or there is an inheritance relationship from the replacement API data type to the deprecated API, we designate it as *SR*, otherwise *DR*. For *Parameters*, we consider the data type and the quantity of parameters. The key for the classification is to match the parameters of the deprecated API and the replacement API. Algorithm 1 presents the process of automatically classifying the migration types on parameters. First we obtain the parameters of the deprecated API and the replacement API (lines 3,4). Then we compare the parameters from the two lists one by one (lines 5-10). In each loop, we get a parameter from *dParams* and determine if there is a parameter equivalent to it in *rParams* (line 6). When there is a correspondence, the two parameters are removed from the both lists (lines 7,8). Finally, the

²<https://javaparser.org>

result is determined by evaluating the two lists (lines 11-22). It returns $P_{unchanged}$ when the two lists are both empty (lines 11-13), which means all the parameters in both lists can be mapped. If $dParams$ is not empty while $rParams$ is empty (lines 14-16), the result is assigned $P_{addition}$. For example, the three tasks in Figure 1 are all classified as $P_{addition}$. The deprecated APIs in TASK1 (i.e., `enable()`) and TASK2 (i.e., `getHours()`) have no parameters while their replacement APIs have parameters. The deprecated API in TASK3 (i.e., `modelToView(1,a)`) has two parameters while its replacement API has an extra parameter (i.e., `Position.Bias.Forward`). If $rParams$ is empty while $dParams$ is not empty (lines 16-18), the result is assigned $P_{reduction}$. The result is assigned P_{change} if both lists are not empty (lines 19-21).

Algorithm 1 Migration Parameters Classify

```

1: function migration_parameters_classify( $dAPI, rAPI$ )
2:    $result \leftarrow P_{unchanged}$ 
3:    $dParams \leftarrow dAPI.getParams()$ 
4:    $rParams \leftarrow rAPI.getParams()$ 
5:   for all  $p \in dParams$  do
6:     if  $\exists q \in rParams, equivalent(p, q)$  then
7:        $dParams.remove(p)$ 
8:        $rParams.remove(q)$ 
9:     end if
10:  end for
11:  if  $dParams.isEmpty() \& \& rParams.isEmpty()$  then
12:     $result \leftarrow P_{unchanged}$ 
13:  end if
14:  if  $dParams.isEmpty() \& \& \neg rParams.isEmpty()$  then
15:     $result \leftarrow P_{addition}$ 
16:  end if
17:  if  $\neg dParams.isEmpty() \& \& rParams.isEmpty()$  then
18:     $result \leftarrow P_{reduction}$ 
19:  end if
20:  if  $\neg dParams.isEmpty() \& \& \neg rParams.isEmpty()$  then
21:     $result \leftarrow P_{change}$ 
22:  end if
23:  return  $result$ 
24: end function

```

The core point in the algorithm is how to determine that two parameters are equivalent. (line 6). We leverage two steps to deal with this concern. The first step is to find the association of the parameters at the source code level. Two parameters can be considered equivalent if they have the same or similar reference locations, i.e., the program statements that reference the parameters. We analyze the method bodies of the deprecated API and the replacement API respectively and find the location of the parameters in their method bodies. When the location is a method call, we further track the method along with its method call chains. We specify two parameters have the same reference locations when they have the same interaction with the same field or they are used in the same method call. If the first step fails to determine the equivalence, the second step is taken. This step makes decision by means of comparing the data types and declaration names of the parameters. They are

considered equivalent when they have the same or inherited data types, and the same declaration names.

Figure 3 illustrates the process of signature analysis. We first separate the return value and parameters from the deprecated API and its replacement API. Then we compare their return value type (RT_x and RT_y) and parameters ($\{param1\}$ and $\{param1, param2\}$) separately. It can be classified as *Different Return* in *Return Type* dimension. From the perspective of *Parameters*, we traverse the parameter list of the deprecated API. $param1$ is removed because it exists in both parameter list (lines 6-9 in Algorithm 1) while $param2$ is left. Therefore, it can be classified as *Parameter Addition* in *Parameters* dimension. Classification results of two dimensions (*Different Return* and *Parameter Addition*) are combined as the result of signature analysis (DR_PA).

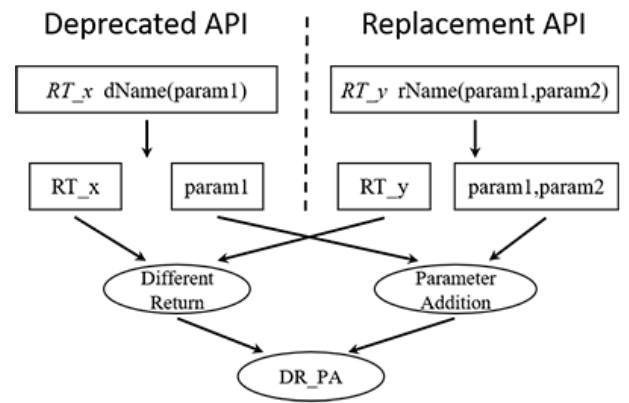


Figure 3: An Example of Signature Analysis Process.

Strategy Synthesis. For a pair of deprecated API and its replacement, the migration strategy is synthesized by the operations covering the dimensions in Table 1. The operation towards the method name is straightforward based on the API names organized in the pattern. The operations to the other elements are instantiated by evaluating the conditions listed in the table. We detailedly explain the rules for deriving the operations and the examples for the operations can be seen on our web page³.

There are three operations that can be computed for caller when the callers of the APIs are not equivalent. The following steps are performed to evaluate the conditions in order.

- When the replacement API can be found on any method call chain of the deprecated API, the condition is specified as *Traceable*. We analyze the caller of the replacement API on one method call chain and collect all the statements involving the caller from its declaration to its usage. The statements are abstracted and are recorded in the operation *setCaller*. It is worth noting that the abstracted statements are to be instantiated according to the context of the replacement API client when migrating.
- When the class to which the caller of the replacement API belongs contains a method (e.g., *constructor* or *getInstance*) to create an instance, the condition is specified as *Instantiable*.

³<https://github.com/cashewnut/deprecated>

Table 2: DiffTypes and Strategies of 412 Migration Patterns. Each cell in **DiffType** represents the number of **DR_PA/SR_PA/DR_PR/SR_PR/DR_PC/SR_PC/DR_PU/SR_PU**. Each cell in **Strategy** represents the number of **Caller (set-Caller/createCaller/annotate)**, **Name(rename)**, **Return Type(convertType/convert/annotate)** and **Parameter (addParam/removeParam/updateParam/setParam/annotate)**.

	Version	DiffType	Strategy			
			Caller	Name	Return Type	Parameter
jdk	1.8	8/56/4/9/20/28/15/109	11/19/1	213	3/6/7	59/51/16/23/19
junit	4.12	0/2/0/0/0/1/0/11	5/0/3	3	0/0/0	2/0/2/0/0
guava	28.0	0/4/0/1/0/6/0/55/	17/1/20	44	5/0/0	2/2/1/85/8
commons-io	2.6	0/27/0/3/0/6/0/8	9/0/0	5	3/0/0	42/5/0/0/0
commons-lang3	3.9	0/0/1/2/7/10/0/19	9/12/0	14	3/2/3	9/12/1/8/10

We use an instantiation method as the statement to create an instance of the class as the caller. The statement is synthesized in the operation *createCaller*.

- (c) When the above conditions are not met, the condition is specified as *Unknown*. We design natural language annotations in specific templates (e.g., *Create an instance for \$class as the caller.*) in the operation *annotate*.

There are three operations for return type when its diff type is *DR*. The following ordered steps are performed.

- (a) When the replacement API can be found in the any method call chain of the deprecated API, the condition is specified as *Traceable*. We collect all the statements involving converting return value from the replacement API to the deprecated API in the method body of the deprecated API, and synthesize the statements in *convertType*.
- (b) When the return types of the deprecated API and the replacement API match any of the type conversion strategies we prepared in advance (e.g., from *String* to *int*), the condition is specified as *Convertible*. The operation *convert* including both types is interpreted as a type cast statement when performing migration.
- (c) When the above conditions are not met, it is specified as *Unknown*. Similarly, annotations in specific templates (e.g., *Different Return Type! Convert \$dType to \$rType.*) are prepared.

There are five operations for each parameter in the parameter list when its diff type is not *PU*.

- (a) For each parameter in the remaining set of *rParams* when the *DiffType* is *PA* from algorithm 1, the condition is specified as *Creation*. The statements or the expression in *addParam* are determined from two aspects. If the deprecated API invokes the replacement API in its body, the statements involving the new parameter from its declaration to its usage are collected as the statements in the operation. Conversely, if the replacement API invokes the deprecated API, we consider the commonly existed situation that the new parameter is referenced in the decision expression of the branch. In this case, we utilize the expression that determines the branch in which the deprecated API is called.
- (b) For each parameter in the remaining set of *dParams* when the *DiffType* is *PR* from algorithm 1, the condition is specified as *Deletion*. The operation is to remove the parameter from the parameter list of the deprecated API.

- (c) For each parameter in the remaining set of *dParams* when the *DiffType* is *PC*, if there is a correspondent parameter in the remaining set of *rParams* and the expressions involving the both parameters have the same or similar reference locations, the condition is specified as *Change*. The operation for *Change* is equivalent to the combination of operations of *Creation* and *Deletion* which is to remove a parameter from the parameter list of the deprecated API and add a new parameter for the replacement API. On the other hand, if the two parameters have the same or inherited data types as well as the same declaration names, the condition is specified as *WeakMap*. The operation for *WeakMap* is to directly set the parameter to the replacement API by *setParam*. However, the confidence of this operation is *M* because there has no actual correspondence at the source level between the two parameters.
- (d) When the above conditions are not met, it is specified as *Unknown*. Similarly, annotations in specific templates (e.g., *Different Parameters! Convert {\$dParam...} to {\$rParam...}*) are given.

3.3 Patterns Generated for Java-based Libraries

We leverage the process to construct the patterns for five Java-based libraries which are *jdk*, *junit*, *guava*, *commons-io* and *commons-lang3*. There are totally 408 deprecated APIs with replacement APIs. Some deprecated APIs have multiple replacement APIs that can be replaced by one of the replacement APIs. For this one-to-many case, we respectively associate the deprecated API with its replacement API. Finally, we get 412 pairs and generate *Migration Pattern* for them.

Table 2 presents the *DiffType* and *Strategy* of 412 *Migration Patterns*. In the *DiffType* part, about one-eighth of the pairs are classified in *DR* from the perspective of *Return Type*. The remaining seven-eighths have the same return type. From the perspective of *Parameters*, 78 pairs are classified in *PC*. One quarter of the pairs

```
MP = < APIVersion = jdk1.8,
    dAPI = JPasswordField.getText(int,int),
    rAPI = JPasswordField.getPassword(),
    MT = < DR_PR, (String,char[]), ({int,int},{})>,
    MS = {convert(String,char[]), rename(getPassword),
        removeParam(int_1), removeParam(int_2)}>
```

Figure 4: An Example of MP.

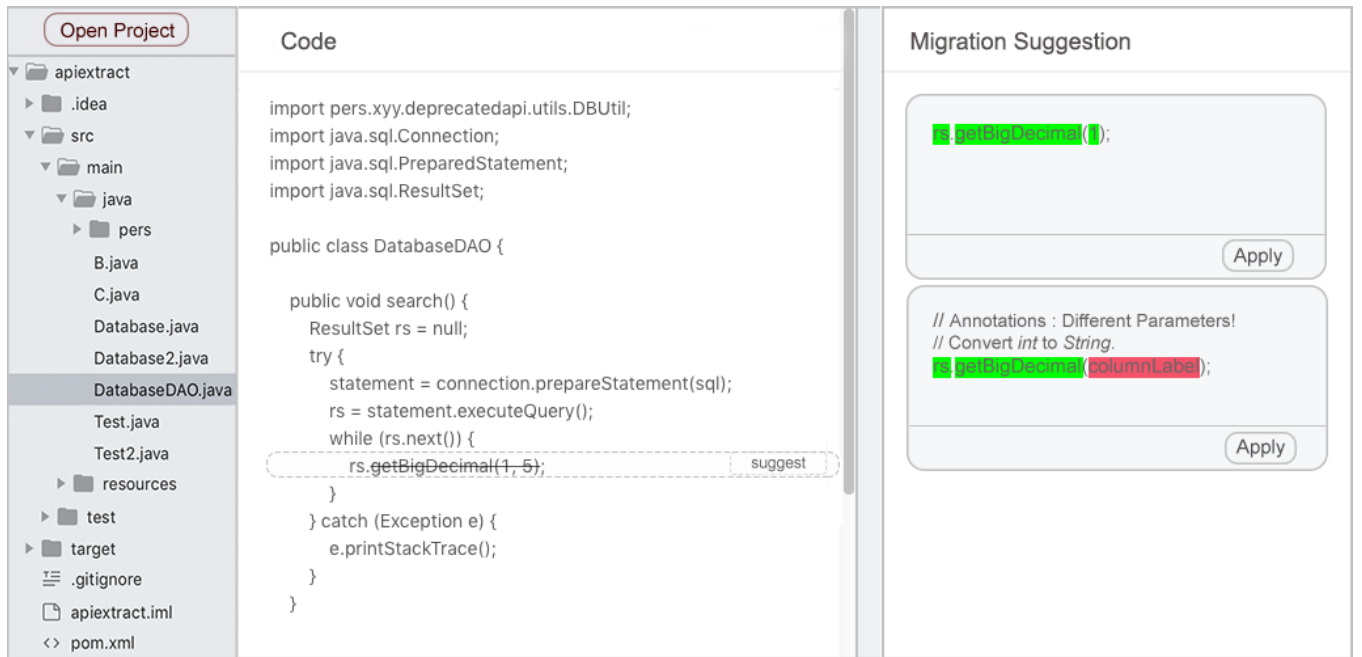


Figure 5: User Interface of DAAMT.

are type *PA* and type *PR* only has 20 pairs. Most of the pairs are classified in *PU*. In the *Strategy* part, about half of the deprecated APIs that need to migrate caller involve *setCaller* operations while 32 deprecated APIs involve *createCaller*. We provide annotations for the rest caller migration. For the operation of return type, 14 deprecated APIs involve *convertType* and 8 involve *convert*. Annotations are provided for the remaining 10 APIs. For the operation of parameters, 114 parameters belonging to deprecated APIs involve *addParam* and 70 parameters involve *removeParam*. Among the rest, 20 operations of *updateParam* and 116 operations of *setParam* are generated. The remaining 37 parameters are suggested by annotations.

We taking a deprecated API (*String JPasswordField.getText(int, int)*) in jdk as an example and describe the process of generating *MP*. According to the description in documentation, the API is deprecated for security reasons and it can be replaced by *char[] JPasswordField.getPassword()*. We can easily obtain *APIVersion*, *dAPI* and *rAPI*. In the *DiffType* part, the deprecated API and its replacement API have different return type. The deprecated API has two parameters while its replacement API has no parameters. From the perspective of *Parameters*, it is classified as *Parameter Reduction* according to Algorithm 1. Therefore, the *DiffType* of this deprecated API is defined as *DR_PR*. In the *Strategy* part, the deprecated API and its replacement API has the same caller so migrating caller is unnecessary. For the *Name*, we use *rename(getPassword)* to change the API name. *convert(String, char[])* is applied to migrate return type. We convert the return type of replacement API via *new String(char[])*. Finally, *removeParam* is applied to the two parameters respectively. Figure 4 presents the *MP* which is generated during this process.

4 DAAMT

Based on the patterns constructed, *DAAMT* facilitates developers to recognize the APIs in Java programs and to migrate these APIs to their replacements. *DAAMT* is now designed as a desktop application. Figure 5 shows its user interface.

A tool user can select a java project and *DAAMT* will subsequently traverse the source files to locate the possible deprecated APIs. If a class file contains a deprecated API, the statement will be marked in the left code box. The user can click the *suggest* button and the tool will provide the migration suggestions in the right box. For the migration task, the tool generates the strategy by synthesizing the operations derived for a specific deprecated API. The strategy is then executed and the suggestions are instantiated in the program context, i.e., specifies the actual return type, parameter name, etc. In particular, if a deprecated API has multiple replacement APIs, there will be multiple suggestion items in the right box. Furthermore, the color of the label indicates the confidence level of the migration operations, i.e., green represents *H*, blue represents *M* and red represents *L*. The selected suggestion can be populated into the original program by clicking the *Apply* button.

5 EVALUATION

We investigate the following research questions to evaluate our approach and tool.

- RQ1. To what extent can this approach support automatic migration of deprecated APIs?
- RQ2. To what extent can this tool help developers to migrate deprecated APIs?

To answer these questions, we leveraged the migration patterns from Table 2 to 6229 open-source projects from github whose stars

Table 3: Experimental Study Results of DiffType.

	DR_PA	SR_PA	DR_PR	SR_PR	DR_PC	SR_PC	DR_PU	SR_PU	Sum
jdk	2/14	762/582	0/6	8/17	0/79	13/41	0/274	1376/4	2161/1017
junit	0/0	0/347	0/0	0/0	0/0	0/0	0/0	33/0	33/347
guava	0/0	1/0	0/0	0/0	0/0	0/8	0/0	704/0	705/8
commons-io	0/0	1335/0	0/0	0/1	0/0	6/1	0/0	2/0	1343/2
commons-lang3	0/0	0/0	3/0	21/0	0/0	3/69	0/0	282/0	309/69

Table 4: Experimental Study Results of Strategies. Each cell is represented as: Caller(setCaller/createCaller/annotate), Name(rename), Return Type(convertType/convert/annotate) and Parameter(addParam/removeParam/updateParam/setParam/annotate).

	Caller	Name	Return Type	Parameter
jdk	127/315/128	2879	45/56/197	836/51/231/115/940
junit	0/0/0	33	0/0/0	347/0/0/0/0
guava	6/70/309	706	0/0/0	1/0/8/0/25
commons-io	3/0/0	4	0/0/0	1335/1/7/0/0
commons-lang3	90/135/0	158	0/3/0	0/24/3/32/507

are greater or equal to 100. Then we conducted a user study to solve specific migration tasks. Both experiments details can be seen on our web page.

5.1 Experimental Study

Description. We assume that a deprecated API is fully equivalent to the replacement API provided in its documentation. Under the assumption, for each selected project, we use *JavaParser* to parse classes and get deprecated APIs by judging whether an API call exists in our prepared migration patterns. Then the migration strategies are applied to these deprecated APIs. According to the completion status of the migration, the results are divided into two types that are named full migration and partial migration. Full migration requires that all elements of the deprecated API can be migrated with confidence *H* or *M* (i.e., all elements of the replacement API already exist in the *MP* before the migration operation). The result is classified as partial migration when any element in the deprecated API is migrated with confidence *L*. Meanwhile, we count the number of different operations used to migrate deprecated APIs.

Result. Table 3 shows the results of the migration for 5994 located deprecated APIs based on the migration patterns of the five java-based libraries separately. Each column represents the migration result for a specific *DiffType*, and each cell lists the number of full migration and partial migration by a slash delimiter. From the table, 4482 (account for 74.76%) deprecated APIs are fully migrated while 1512 (account for 25.24%) deprecated APIs are commented by natural language migration suggestions. The migration depends on the quality of the code documentation. For example, we can fully migrate about two-thirds of deprecated APIs in type *PA* while the rest of them are partial. It indicates that there are still many new parameters that cannot be obtained from the source level in specific APIs like *jdk1.8* and *junit*. We further find that all the deprecated APIs in type *PU* that cannot be fully migrated are different from the replacement APIs in return type or caller. There are 198 deprecated APIs in type *PC* that cannot be fully migrated. 79 of them are due

Table 5: User Study Results.

	Group	Time(Avg/Max/Min)	Accuracy(Avg/Max/Min)
TAKS1	EG	7min39s/11min37s/3min38s	92.74%/100%/87.1%
	CG	7min54s/15min30s/4min8s	92.74%100%87.1%
TAKS2	EG	1min53s/3min7s/48s	93.1%/100%/88.89%
	CG	6min18s/18min16s/2min32s	89.8%/92.6%/85.5%
TAKS3	EG	6min4s/13min13s/1min59s	93.2%/100%/86.4%
	CG	7min31s/13min21s/3min45s	92.5%/100%/86.4%
TAKS4	EG	12min38s/20min0s/3min31s	94.1%/100%/88.2%
	CG	13min51s/20min0s/6min53s	93.4%/94.1%/88.2%
TAKS5	EG	4min49s/13min8s/1min27s	98.95%/100%/95.8%
	CG	5min54s/10min0s/3min30s	98.95%/100%/98.95%

to the different return type and the rest are because the statements or expression of operations contain members that we cannot get, such as fields declared as *private*. There are only 56 deprecated APIs in type *PR* and 25 of them are not fully migration because they have different return type or caller than their replacement APIs. In addition, most of the deprecated APIs in type *DR* are not fully migrated, which indicates that our approach is still limited for some return type conversion. The conversion of the return type requires some additional information support that we have not analyzed, such as other methods in the class which the deprecated API or the replacement API belongs to (e.g., *Calendar.getTime()*). On the other hand, the type conversion strategies we prepared in advance do not conclude enough type conversions. Table 4 shows the number of different operations used to migrated deprecated APIs. Only 223 deprecated APIs that migrate caller involve *setCaller* operations while 520 deprecated APIs involve *createCaller*. We provide annotations for the rest caller migration. For the operation of name, a total of 3747 deprecated APIs that migrate name involve *rename*. For the operation return type, the deprecated APIs of the selected libraries has hardly changed the return type except *jdk*. 45 deprecated APIs involve *convertType* and 59 involve *convert*. Annotations are provided for the remaining 197 APIs. For the operation parameter, the most (2519) parameters involve *addParam*. Only 76 parameters involve *removeParam* and 243 parameters involve *updateParam*. There are 145 parameters that match the condition *WeakMap* which can be migrated by *setParam*. We provide annotations for the remaining 1427 parameters. Overall, the results answer *RQ1* that our tool can support the automatic migration for most of the deprecated APIs.

5.2 User Study

Description. We invited sixteen developers who have used Java for one to six years and divide them into *Experiment Group (EG)* and *Control Group (CG)*. Each group had eight participants and their development experience was evenly distributed. We designed five

code snippets (see from our web page), each of which contained five to seven deprecated APIs. The task of *EG* was to migrate these deprecated APIs using *DAAMT*. They could search from documentation if a deprecated API could not be completely migrated. The task of *CG* was to migrate these deprecated APIs depending on documentation only. All deprecated APIs needed to be migrated into their replacements specified in the documentation and these code snippets should be successfully compiled after migration. We separately recorded the time cost by the participants for each code snippet and calculated the accuracy. The time was recorded as 20 minutes if a participant completed a task for more than 20 minutes or gave up the task. Accuracy represents the percentage of elements that were successfully migrated among the all elements which should be migrated.

Result. Table 5 shows the time and accuracy of the user study. In *TASK1*, the average time cost by *EG* is similar to *CG* because some of the deprecated APIs (e.g., *date.setYear(int)*) could only be partially migrated. Our approach can migrate *date.setYear(int)* to *calendar.set(Calendar.YEAR,int)*. In order to make the replacement API conform to the context, participants in *EG* have to search for a method to convert *Calendar* to *Date*, which takes a certain amount of time. Both groups have the same accuracy in *TASK1*. *TASK2* shows the most difference. The reason is that *DAAMT* can almost completely migrate the deprecated API for *TASK2*, and the participants did not need additional actions. The accuracy of *EG* in *TASK2* is significantly higher than that of *CG*. The reason is that there is a deprecated API in *TASK2* whose replacement API has the similar signature with another API in its replacement message (*toURL()* and *toURI()*), causing the participants of *CG* to find the wrong replacement API. In the remaining three tasks, the average time cost by *EG* is less than *CG* for at least one minute and the accuracy of *EG* is also slightly higher than that of *CG*. Overall, *EG* performs better than *CG*. The results answer *RQ2* that *DAAMT* can improve the efficiency and accuracy of the tasks compared with the manual migration.

5.3 Threats to Validity

The major threat is that our experimental study and user study only involved a limited number of open-source libraries and participants. Our approach and evaluation might not generalize to libraries in other domains and a broader variety of users.

In the experimental study, the 5994 deprecated APIs used to evaluate our approach are all derived from the real-world Java projects in github whose stars are greater or equal to 100. A potential threat is that our experimental study are only based on 5 Java libraries with specific documentation. Therefore, it is not clear whether the results of experimental study may differ if more libraries of libraries with relatively low quality documentation are considered. Another threat is that the 5 libraries and clients we choose are open-source projects. So we are not sure whether our approach has the same result for commercial projects.

In the user study, each group has 8 participants and their development experience is evenly distributed. We think that development experience is positively related to the time they participate in the development. However, development experience may be determined by a variety of factors. Another threat is that we only design 5 tasks,

which may increase the accidentality of result. Moreover, the time of participants in *EG* learn how to use *DAAMT* is short (5 minutes). We believe users can migrate deprecated APIs more efficiently if they spend more time learning it.

5.4 Discussion

There are still some limitations in our work. At the API signature level, we define the difference between the deprecated API and the replacement API as eight types. From the current data set, all the differences belong to these types. However, we cannot ensure that these eight types are enough to cover all situations. In future work, if we find situations that cannot be covered in a larger data set, we will manually analyze and consider whether to add new types.

Another limitation is that our work is based on the assumption that a deprecated API is equivalent to the replacement API provided in its documentation. However, The assumption cannot be guaranteed in some libraries with low quality documentation. In this case, when a deprecated API is migrated, the consistency of client cannot be guaranteed. In the future, we will try to combine the structural analysis and data based mining to verify the equivalence of the deprecated API and the replacement API. For the case where the assumption is not valid, we can combine some other technologies (e.g., history changed matching) to generate migration strategies.

6 RELATED WORK

Existing API migration techniques is mainly based on historical code change or API documentation. We consider related work from these two classes.

Historical Code Change Henkel et al. [6] proposed an Eclipse plugin called *CatchUp!*. The technique is based on a simple idea: record how developers migrate APIs in the IDE and replay the refactorings to update client code automatically. However, this approach takes resources to record the migration operation and a manual migration is required for recording. Perkins et al. [15] proposed an approach called *method inlining*. They used the method body of the deprecated API to instead of the deprecated API. But the approach dose not remove the adverse effects of the deprecated API on the clients and it is not the solution that API consumers want. Xing et al. [20] implemented the *Diff-CatchUp* tool, which can automatically extract the API changes of the reused framework and provide replacements with the outdated API. These methods only need to analyze a single version of the client and framework but their flaws are obvious.

Schäfer et al. [17] proposed an approach to mine framework usage change rules from the source code of client applications. Degenais et al. [5] proposed a client-server tool named *SemDiff*, which suggests adaptations to client programs by analyzes how a framework adapts to its own changes. Meng et al. [12] introduced *HiMa*, a novel approach which is based on matching recorded in evolution history and aggregating revision-level rules to construct framework-evlution rules. Nguyen et al. [14] presented a tool named *LibSync* which uses graph-based techniques to learn API usage adaptation patterns from other clients that already migrated to a new library version. Wu et al. [19] presented *AURA*, a hybrid approach which combines call dependency and text similarity analysis to keep client systems up-to-date. Hora et al. [7] presented

apiwave, which is used to support keeping track of API evolution and popularity. However, the limitation of these approaches is that they need to get multiple versions of clients or frameworks.

API Documentation Strobl et al. [18] indicated the migration for the deprecated API can be partially automated for most Java APIs. They implemented two tools. The first one is *JaSCUT* which migrates the source code based on a configuration file and the others is *Generator* which heuristically figures out how to migrate deprecated APIs and generates the configuration file. Their work is most relevant to us. But their tools cannot migrate parameters. Our tools use different colors to indicate the confidence of element, which can help developers determine whether they need double-check. The limitation of these approaches is that they rely on specific documentation. Cossette et al. [4] indicated that replacement recommendations in deprecated messages are often missing or incomplete. Ko et al. [9] investigated the quality of the deprecated API documentation and found 39% of the deprecated messages do not provide replacement APIs. Their research shows that deprecated APIs with high quality documentation are easier to migrate but most of the documentation do not contain concrete API usages to help developers use new APIs. Brito et al. [2] presented a large-scale analysis on Java systems and found 36% of the API elements are deprecated without replacement messages. The empirical investigation reveals a tool is needed to recommend deprecation message. It seems that approaches based on historical code change can migrate more deprecated APIs. However, Lamothe et al. [10] found that approaches based on documentation are more effective.

7 CONCLUSION

This paper presented an approach for migrating deprecated APIs with documented replacement. First, we propose the definition of API migration pattern. Then, we design and implement the automated process for generating migration patterns. Based on these patterns, we developed *DAAMT* to provide migration suggestions for API clients.

Limitations of the approach is admitted. First, our approach works only for deprecated APIs with documented replacements, ignoring those without the complete replacement messages. However, the approach can be regarded as general so long as the API pair is provided. We assume the approach can be expanded by leveraging the results of techniques (e.g., history changed matching or data based mining) that can infer the replacement APIs for the deprecated API without documented replacements. Second, our approach still fails to generate migration operations for elements that cannot be determined related on source code level. We assume it can be improved by advanced code analysis techniques.

As for the future work, we first plan to solve the limitations mentioned above by investigating techniques from the areas of source code mining and analysis. Then we plan to migrate *DAAMT* to IDE platform as a plugin to bring greater convenience to developers. Finally, in order to verify the availability and effectiveness of our approach in a wider range of libraries and larger client projects, we plan to construct migration patterns covering more Java-based libraries, and organize larger-scale experiments utilizing the patterns upon clients from a wider range of sources.

8 ACKNOWLEDGMENT

This work was supported in part by the National Key Research and Development Program of China (2016YFB1000801).

REFERENCES

- [1] Christopher Bogart, Christian Kästner, James D. Herbsleb, and Ferdian Thung. 2016. How to break an API: cost negotiation and community values in three software ecosystems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13-18, 2016*. 109–120.
- [2] Gleison Brito, André C. Hora, Marco Tulio Valente, and Romain Robbes. 2016. Do Developers Deprecate APIs with Replacement Messages? A Large-Scale Analysis on Java Systems. In *IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER 2016, Suita, Osaka, Japan, March 14-18, 2016 - Volume 1*. 360–369.
- [3] Gleison Brito, André C. Hora, Marco Tulio Valente, and Romain Robbes. 2018. On the use of replacement messages in API deprecation: An empirical study. *Journal of Systems and Software* 137 (2018), 306–321.
- [4] Bradley Cossette and Robert J. Walker. 2012. Seeking the ground truth: a retrospective study on the evolution and migration of software libraries. In *20th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-20), SIGSOFT/FSE'12, Cary, NC, USA - November 11 - 16, 2012*. 55.
- [5] Barthélémy Dagenais and Martin P. Robillard. 2008. Recommending adaptive changes for framework evolution. In *30th International Conference on Software Engineering (ICSE 2008), Leipzig, Germany, May 10-18, 2008*. 481–490.
- [6] Johannes Henkel and Amer Diwan. 2005. CatchUp!: capturing and replaying refactorings to support API evolution. In *27th International Conference on Software Engineering (ICSE 2005), 15-21 May 2005, St. Louis, Missouri, USA*. 274–283.
- [7] André C. Hora and Marco Tulio Valente. 2015. Apiwave: Keeping track of API popularity and migration. In *2015 IEEE International Conference on Software Maintenance and Evolution, ICSME 2015, Bremen, Germany, September 29 - October 1, 2015*. 321–323.
- [8] Ralph E Johnson and Brian Foote. 1988. Designing reusable classes. *Journal of object-oriented programming* 1, 2 (1988), 22–35.
- [9] Deokyeon Ko, Kyeongwook Ma, Sooyong Park, Suntae Kim, Dongsun Kim, and Yves Le Traon. 2014. API Document Quality for Resolving Deprecated APIs. In *21st Asia-Pacific Software Engineering Conference, APSEC 2014, Jeju, South Korea, December 1-4, 2014. Volume 2: Industry, Short, and QuASoQ Papers*. 27–30.
- [10] Maxime Lamothe and Weiyi Shang. 2018. Exploring the use of automated API migrating techniques in practice: an experience report on Android. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR 2018, Gothenburg, Sweden, May 28-29, 2018*. 503–514.
- [11] Li Li, Jun Gao, Tegawendé F. Bissyandé, Lei Ma, Xin Xia, and Jacques Klein. 2018. Characterising deprecated Android APIs. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR 2018, Gothenburg, Sweden, May 28-29, 2018*. 254–264.
- [12] Sichen Meng, Xiaoyin Wang, Lu Zhang, and Hong Mei. 2012. A history-based matching approach to identification of framework evolution. In *34th International Conference on Software Engineering, ICSE 2012, June 2-9, 2012, Zurich, Switzerland*. 353–363.
- [13] Simon Moser and Oscar Nierstrasz. 1996. The Effect of Object-Oriented Frameworks on Developer Productivity. *IEEE Computer* 29, 9 (1996), 45–51.
- [14] Hoan Anh Nguyen, Tung Thanh Nguyen, Gary Wilson Jr., Anh Tuan Nguyen, Miryung Kim, and Tien N. Nguyen. 2010. A graph-based approach to API usage adaptation. In *Proceedings of the 25th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2010, October 17-21, 2010, Reno/Tahoe, Nevada, USA*. 302–321.
- [15] Jeff H. Perkins. 2005. Automatically generating refactorings to support API evolution. In *Proceedings of the 2005 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis For Software Tools and Engineering, PASTE'05, Lisbon, Portugal, September 5-6, 2005*. 111–114.
- [16] Anand Ashok Sawant, Romain Robbes, and Alberto Bacchelli. 2018. On the reaction to deprecation of clients of 4 + 1 popular Java APIs and the JDK. *Empirical Software Engineering* 23, 4 (2018), 2158–2197.
- [17] Thorsten Schäfer, Jan Jonas, and Mira Mezini. 2008. Mining framework usage changes from instantiation code. In *30th International Conference on Software Engineering (ICSE 2008), Leipzig, Germany, May 10-18, 2008*. 471–480.
- [18] Roman Strobl and Zdenek Tronicek. 2013. Migration from deprecated API in Java. In *Conference on Systems, Programming, and Applications: Software for Humanity, SPLASH '13, Indianapolis, IN, USA, October 26-31, 2013 - Companion Volume*. 85–86.
- [19] Wei Wu, Yann-Gaël Guéhéneuc, Giuliano Antoniol, and Miryung Kim. 2010. AURA: a hybrid approach to identify framework evolution. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1, ICSE 2010, Cape Town, South Africa, 1-8 May 2010*. 325–334.
- [20] Zhenchang Xing and Eleni Stroulia. 2007. API-Evolution Support with Diff-CatchUp. *IEEE Trans. Software Eng.* 33, 12 (2007), 818–836.